```
show_regexp('Price $12.', /[^A-Z]/)      # => P->r<-ice $12.
show_regexp('Price $12.', /[^\w]/)       # => Price-> <-$12.
show_regexp('Price $12.', /[a-z][^a-z]/) # => Pric->e <-$12.
```

Some character classes are used so frequently that Ruby provides abbreviations for them. These abbreviations are listed in Table 2, Character class abbreviations, on page 136—they may be used both within brackets and in the body of a pattern.

```
show_regexp('It costs $12.', /\s/) # => It-> <-costs $12.
show_regexp('It costs $12.', /\d/) # => It costs $->1<-2.
```

If you want to include the literal characters] and - in a character class, escape them with a backslash:

```
a = 'see [The PickAxe-page 123]'
show_regexp(a, /[\]]/)    # => see [The PickAxe-page 123->]<-
show_regexp(a, /[0-9\]]/) # => see [The PickAxe-page ->1<-23]
show_regexp(a, /[\d\-]/)  # => see [The PickAxe->-<-page 123]
```

Finally, a period (.) appearing outside brackets represents any character except a newline (though in multiline mode it matches a newline, too):

```
a = 'It costs $12.'
show_regexp(a, /c.s/) # => It ->cos<-ts $12.
show_regexp(a, /./)   # => ->I<-t costs $12.
show_regexp(a, /\./)  # => It costs $12->.<-
```

Repetition

Back in Working with Strings, on page 123, we specified the pattern that split the song list line, /\s*\|\s*/, and we said we wanted to match a vertical bar surrounded by an arbitrary amount of whitespace. We now know that the \s sequences match a single whitespace character and \| means a literal vertical bar, so it seems likely that the asterisks somehow mean "an arbitrary amount." In fact, the asterisk is one of a number of modifiers that allow you to match multiple occurrences of a pattern.

If *r* stands for the immediately preceding regular expression within a pattern, then

*r**	Matches zero or more occurrences of *r*
r+	Matches one or more occurrences of *r*
r?	Matches zero or one occurrence of *r*
r{m,n}	Matches at least m and at most n occurrences of *r*
r{m,}	Matches at least m occurrences of *r*
r{,n}	Matches at most n occurrences of *r*
r{m}	Matches exactly m occurrences of *r*

These repetition constructs have a high precedence—they bind only to the immediately preceding matching construct in the pattern. /ab+/ matches an *a* followed by one or more *b*'s, not a sequence of *ab*'s. If you want a sequence of *ab*'s, you need to group the pattern, /(ab)+/.

These patterns are called *greedy*, because by default they will match as much of the string as they can. You can alter this behavior and have them match the minimum by adding a question mark suffix. The repetition is then called *lazy*—it stops once it has done the minimum amount of work required.